DIGITAL TWIN · FORECASTING · CONTROL SCIENCE

Sending 5G traffic to the right box *before* it jams

A digital twin of a national 5G user plane, a demand forecaster, an optimizer — and 588 head-to-head experiments to find out whether the smart version actually wins.

16 M	24	96	112	588	0
SIMULATED USERS	UPF BOXES	TRAFFIC GROUPS	DAYS SIMULATED	CONTROLLER TESTS	UNSAFE RELEASES

HOW THIS DECK IS ORGANISED

Four parts

00

THE BIG PICTURE

What we built, and what we learned

A UPF is the box that carries 5G user traffic. When a phone starts a session, the network picks which UPF it attaches to. That pick is the only thing we can control. The question: does predicting traffic make that pick better than a simple fixed rule?

THE PROBLEM

One box, one lever

WHAT A UPF IS

A UPF — User Plane Function — is the box that carries 5G user traffic. Every video, call and download passes through one.

If a UPF gets too busy, traffic queues up, and then it gets dropped. That is the failure we are trying to avoid.

When a phone starts a new session, a component called the SMF decides which UPF it attaches to.

WHAT WE CAN AND CANNOT CHANGE

We can change where NEW sessions are sent. That is the entire lever.

We cannot move a session once it is running. Whether C-DOT's system can do that at all is still an open question — and it turns out to be the most important one in this whole project.

Sessions last hours. So most of the load on a box at any moment was placed there long ago, and cannot be taken back.

THE QUESTION THIS PROJECT ANSWERS

Can predicting how much traffic is coming let us place new sessions better than a simple capacity-proportional rule — well enough, and safely enough, to deploy?

HOW THE SYSTEM WORKS

The loop, end to end

The system runs once every 10 minutes. The most important rule is that no stage is allowed to see the future.

STEP 1

Simulate

Generate 30 seconds of network activity: who starts a session, how long it lasts, how much bandwidth it uses, which boxes are healthy.

STEP 2

Clean up

Turn raw counters into usable measurements. Throw away readings that are broken — resets, gaps, duplicates, stale samples.

STEP 3

Predict

For each traffic group, forecast demand 10 to 80 minutes ahead. Give a best guess and a safe upper bound.

STEP 4

Decide

An optimizer works out what share of new sessions each box should receive, given the forecast and the capacity limits.

STEP 5

Check & apply

A separate checker validates the plan. If anything looks wrong, fall back to the simple rule instead.

WHY “NO PEEKING” MATTERS

Telemetry arrives in 30-second buckets, and a bucket is only usable once it closes. Every input feature carries a timestamp, and training discards anything that was not yet available when the forecast was made. A decision made at time t can only affect sessions starting after t. Without this rule, a model can quietly cheat and look far better than it really is.

THE HEADLINE RESULT

The platform works. The smart controllers do not.

Keeping those two findings apart is the point of the whole project.

PASS	PASS	11.6%	NO	NO	SIMPLE
SIMULATOR ENGINEERING	RUNS ON 12 CLUSTER NODES	BEST FORECAST GAIN TARGET WAS 15%	MPC CONTROLLER NOT RELEASED	PRE-DRAIN NOT RELEASED	THE RULE WE STILL USE

WHAT WORKED

The traffic adds up exactly
Offered = carried + dropped + queued + rejected. Gap: 0.0 bytes.

Memory stays flat
Simulating 7× longer uses only 4.1% more memory.

Everything is reproducible
Every output hashed, inputs frozen, nothing silently overwritten.

Session-length estimation works
Learned from ordinary logs; survived five unknown distributions.

There is room to improve
A version that knows the future removes 100% of overload.

Nothing unsafe shipped
588 tests; no controller released without passing every check.

WHAT DID NOT

The better forecast models
LightGBM improved accuracy 11.6%. We needed 15%.

The linear-program controller
Cut overload 2.4%. We needed 20%. Tuning made it worse.

The MPC controller
Break-even in development; 13.3% WORSE on 128 realistic test days.

Transfer between situations
Pre-drain gained 79–83% on planned outages, lost 5.6–7.2% on surprises.

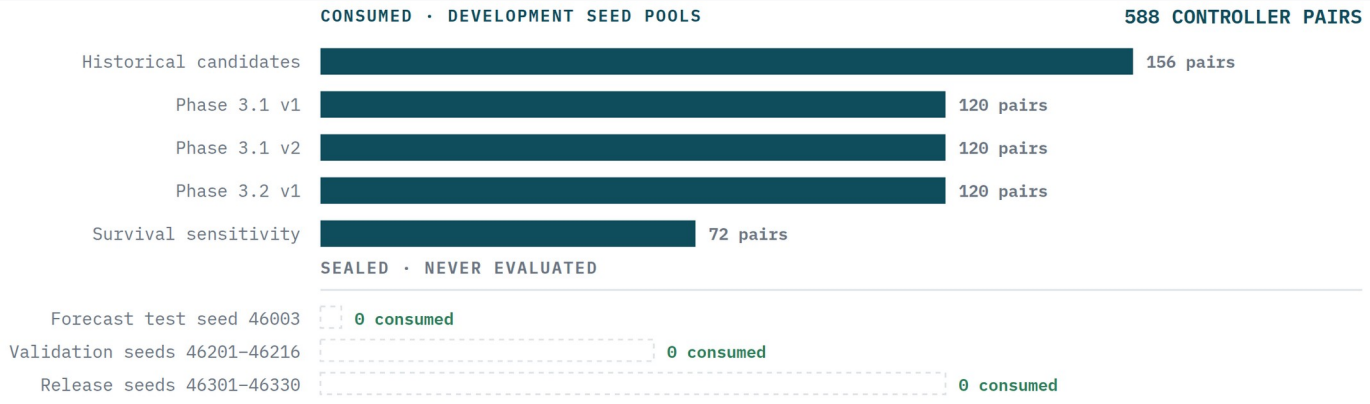
Worst-case safety
Best candidate failed the worst-day limit by 0.37 points.

Decision speed
225–970 ms per decision; only one candidate met the 500 ms budget.

METHOD

How we decided what counts as a win

Every experiment was written down before it ran, with a fixed list of pass/fail checks. Test data sits in three separate pools, and a controller only reaches the next pool by passing every check on the current one.



- **One main number, chosen up front**
  Uplink overload area: how far above the safe limit a box goes, times how long it stays there.
- **Every comparison is like-for-like**
  Same day, same random numbers, same failures. Any difference is the controller alone.
- **Error bars are compulsory**
  Average gain, a 95% confidence range, a severity-weighted total, and the single worst day.
- **When in doubt, stop**
  Stale data, uncertain telemetry or an infeasible plan all hand control back to the simple rule.

01

THE SIMULATOR

Building a copy of a national 5G network

We do not simulate individual data packets — far too slow at this size. We simulate sessions: one starts, uses bandwidth for a while, then ends. We track how much each box is carrying and whether it exceeds its safe limit. Fast enough to run 112 simulated days.

THE ENGINE

What happens in one 30-second step

Each mechanism draws from its own stream of random numbers, so changing the failure schedule cannot accidentally change which users arrive. Twenty steps — 10 minutes — make one decision round.

01 **Time of day.** Daily and weekly cycles. Home areas peak in the evening, business areas mid-day, 7–11 hours apart. People move between zones; the total headcount is conserved exactly.

02 **Scheduled events.** A stadium match or planned maintenance is applied only after the moment the controller was actually told about it.

03 **Random variation.** A slow drift where each step partly carries over from the last, plus occasional bursts of random length.

04 **New sessions arrive.** Drawn from a Poisson process whose rate changes over the day.

05 **Session size and length.** Duration from a heavy-tailed distribution — most short, a few very long. Uplink and downlink bandwidth drawn together, so they are correlated.

06 **Placement.** Each new session goes to a box chosen by whichever rule is active. This is the only lever in the whole system.

07 **Capacity check.** Apply failures, then count traffic offered, carried, queued, dropped and rejected — separately for uplink and downlink.

08 **Emit telemetry.** Write counters out, including flags for resets, restarts, and missing or stale readings.

STRUCTURE

How traffic is grouped

Demand is split into 96 traffic groups: 8 area types × 12 service classes. Each has its own arrival rate, session length, bandwidth profile and list of boxes it may use. The forecaster predicts per group; the optimizer decides per group.

ITEM	VALUE	DETAIL
Area types	8	four urban, plus industrial, airport, stadium and rural — detailed next slide
Service types	12	consumer internet, video, voice; enterprise VPN; industry telemetry & control; vehicles; public safety; stadium media; IoT; roaming
Traffic groups	96	8 area types × 12 service classes
UPF boxes	24	16 edge, 4 regional, 4 central; each group may use 6 of them
Edge box capacity	2.6 / 5.2 Tbps	uplink / downlink, with 78% / 80% treated as the safe operating limit
Session limit	170 000	per box, with 82% as the safe limit
Network delay	7 / 22 ms	staying in your own zone vs crossing into another
Simulated users	16 000 000	modelled as population counts, not individual subscriber identities

The capacity numbers are assumptions we chose. C-DOT has not published real per-box capacity, so these are placeholders until we get the real figures.

AREAS

The eight area types we model

The simulator does not model geography as a map. It models eight area archetypes, each with its own population scale and its own mix of services. A traffic group is one service class inside one area.

AREA	WHAT IT REPRESENTS	SCALE Z(K)	SERVICES THAT DOMINATE	DAILY SHAPE
north-urban	Dense residential	0.70	Consumer video, social, gaming	Evening peak around 20:00
south-urban	Dense residential	0.80	Consumer video, social, gaming	Evening peak around 20:00
east-urban	Mixed residential / retail	0.90	Consumer video, voice, enterprise	Broad evening peak
west-urban	Mixed residential / retail	1.00	Consumer video, voice, enterprise	Broad evening peak
industrial	Factory / logistics estate	1.10	Industrial URLLC, massive IoT, edge AI	Flat-ish, day-shift bias
airport	Transport hub	1.20	Roaming, voice, video conference, V2X	Twin commute peaks
stadium	Large-event venue	1.30	Social / live upload, video, voice	Idle, then extreme event spikes
rural	Sparse coverage	1.40	Voice, massive IoT, consumer video	Low amplitude, flat

AN HONEST LIMITATION OF THE CURRENT BUILD

The area scale is a straight arithmetic ramp — $z(k) = 0.70 + 0.10k$ over the zone index — not a figure calibrated per area type. That is why “rural” carries the largest multiplier: it is last in the list, not busier than a city. The area label drives which services and which daily curve apply; it does not yet drive a realistic population. Replacing this ramp with real per-area subscriber counts is one of the first things operator data would fix.

AREAS

How the shape of a day differs by area

Each service class carries its own daily curve, built as a sum of wrapped Gaussian peaks. The area decides which classes are present and how many; the class decides when they are busy.

DAILY PROFILE	CURVE	CLASSES THAT USE IT	TYPICAL AREA
Evening	0.42 + 1.18 · peak(20:00, 3.2 h)	Consumer video, social / live	Residential areas
Late evening	0.45 + 1.25 · peak(22:00, 3.8 h)	Gaming	Residential areas
Business	0.28 + 1.32 · peak(13:00, 4.0 h)	Enterprise, video conference, edge AI	Commercial areas
Industrial	0.55 + 0.90 · peak(11:00, 5.2 h)	Industrial URLLC	Factory estates
Commute	0.38 + 0.88 · peak(08:00) + 1.00 · peak(18:00)	IMS voice, connected vehicle	Airport, transit
Overnight	0.30 + 1.35 · peak(02:00, 2.8 h)	Cloud backup	Everywhere
Flat	0.92 + 0.08 · sin(2π(h − 6)/24)	Massive IoT, public safety	Everywhere

WHY THE PEAKS MATTER

The business curve bottoms out at 0.28 and peaks at 1.60 — a 5.7× swing across the day. The flat IoT curve moves between 0.84 and 1.00, barely 1.2×.

Because residential and business peaks sit 7 hours apart, the national total never sees one sharp spike. That is exactly why a static capacity split survives so well: the load moves around, but the aggregate stays smooth.

WHAT MULTIPLIES IT

On top of the daily curve sit three multipliers: a per-service weekend factor (0.45 for enterprise up to 1.35 for social), independent weekly noise drawn from U[0.86, 1.16], and any active surge.

Surges arrive 12 times a week, multiply an area's traffic by 2.5× to 8.0×, and last 2 to 10 hours.

TRAFFIC CLASSES

The twelve categories of 5G traffic

Each class carries a 3GPP 5QI value, its own data network name and slice identifier, its own session length, and its own uplink/downlink balance. Together they span the three families 5G is designed for: broadband, ultra-reliable low-latency, and massive machine-type.

CLASS	5QI	DATA NETWORK	SESSION LENGTH	UL / DL MBPS	DAILY PROFILE	WEEKEND	EXAMPLE
Consumer video	9	internet-video	1–12 h	0.15 / 4.00	Evening	1.25	Streaming to a phone
Social / live	8	social-live	10 m–2 h	2.00 / 1.00	Evening	1.35	Uploading from a crowd
Gaming	3	gaming	40 m–6 h	0.20 / 0.40	Late	1.30	Latency-sensitive play
IMS voice	1	ims-voice	10 m–4 h	0.08 / 0.08	Commute	0.90	Carrier voice calls
Enterprise	7	enterprise	30 m–8 h	1.00 / 2.00	Business	0.45	Office VPN and web
Video conference	2	enterprise-rtc	30 m–6 h	1.50 / 1.50	Business	0.50	Symmetric real-time
Industrial URLLC	82	factory-control	4–24 h	0.10 / 0.08	Industrial	0.75	Machine control loops
Massive IoT	9	mmtc	6–48 h	0.020 / 0.004	Flat	1.00	Meters and sensors
Connected vehicle	84	v2x	5 m–1 h	0.50 / 1.00	Commute	0.80	V2X telemetry
Edge AI	7	edge-inference	10 m–2 h	3.00 / 6.00	Business	0.70	Inference at the edge
Cloud backup	9	enterprise-backup	1–12 h	8.00 / 2.00	Overnight	1.15	Bulk upload
Public safety	65	mission-critical	20 m–3 h	2.00 / 2.00	Flat	1.00	Mission-critical push

Eight areas × twelve classes = 96 traffic groups. The steering key is (area, data network, slice); 5QI is carried as descriptive metadata because it is not assumed to be a UPF-selection key. Base arrival rates run from 22.4 sessions per 30 s (cloud backup) to 448 (massive IoT) per group.

How traffic is generated, mathematically

Every quantity below is drawn from a named distribution with a seeded, independent random stream. Same manifest, same seed, same result — bit for bit.

AS-BUILT EXTREME GENERATOR · THE 16-WEEK TRAINING CORPUS

Session arrivals
New sessions in one 30-second step, for one traffic group.

$$N_{g,t} \sim \text{Poisson}(\lambda_g \cdot a_g(t))$$

Sampled exactly by summing independent Poisson chunks — no Gaussian approximation.

Base rate per group
The service's own rate, scaled by area and by the national factor.

$$\lambda_g = \lambda_s \cdot z(k) \cdot S \quad , \quad z(k) = 0.70 + 0.10k \quad , \quad S = 16$$

s = service class, k = area index 0…7, S = the ×16 national scale factor.

Arrival factor
Everything that modulates the base rate, recomputed once per simulated hour.

$$a_g(t) = D_s(h) \cdot W_s(d) \cdot v_{g,w} \cdot M(t)$$

Held constant for all 120 ticks in that hour.

Daily calendar curve
A sum of wrapped Gaussian peaks — one per daily rush for that service.

$$D_s(h) = \alpha_s + \Sigma_j \beta_j \exp(-\tfrac{1}{2}(\delta(h, c_j) / w_j)^2)$$

δ(h, c) = min(|h − c| , 24 − |h − c|) — circular distance, so midnight wraps correctly.

Weekly noise
Independent per group, per week.

$$v_{g,w} \sim U[0.86, 1.16]$$

Surge multiplier
Active flash-crowd events multiply the affected groups.

$$M(t) = \Pi_{e \in E(t)} m_e \quad , \quad m_e \sim U[2.5, 8.0]$$

12 surges per week, each lasting 2–10 hours.

Session lifetime
Drawn once when the session is admitted.

$$L_g \sim \text{DiscreteUniform}\{ L_{min}(s) , L_{max}(s) \}$$

In 30-second steps. From 5 minutes (connected vehicle) to 48 hours (massive IoT).

Offered load on a box
Sum over every session the box is currently carrying.

$$R^{\text{UL}}{}_u(t) = \Sigma_g n_{g,u}(t) \cdot r^{\text{UL}}{}_s$$

Every session of a class carries the same fixed rate here — which is why session count, uplink and downlink are perfectly correlated.

Steady-state occupancy
Little's Law — the identity we use to sanity-check the run.

$$E[A] = \bar{\lambda} \cdot E[L] \rightarrow 453.21 \times 29\,628 = 13.43\ \text{M}$$

Measured active sessions were 13 422 319 — 0.04% from the analytic value.

THE MODEL

The realism layer added on top

The 16-week corpus above uses fixed per-class rates and uniform session lengths. traffic-model/2.0 replaces those with correlated, heavy-tailed processes — and every one is re-fitted from the simulator's own output to check it came out right.

TRAFFIC-MODEL/2.0 · THE ADDED REALISM LAYER

Latent demand drift
An AR(1) process, so demand carries momentum instead of jumping independently.

$$x_{g,t} = \varphi_g\, x_{g,t-1} + \varepsilon_t \quad , \quad \varepsilon_t \sim N(0, \sigma_g^2)$$

φ ∈ [0.72, 0.94], σ ∈ [0.03, 0.12]. Refitted from output: 0.8198 against a configured 0.8200.

Burst state
A two-state Markov chain with a heavy-tailed dwell time in the ON state.

$$B_t \in \{0, 1\} \quad , \quad P(0 \to 1) = p_{in} \quad , \quad P(1 \to 0) = p_{out}$$

Measured dwell: median 8 steps, p90 26, p99 48. Multiplier capped at 4.0×.

Session lifetime
Bounded Pareto or lognormal — most sessions short, a few very long.

$$f(\ell) \propto \ell^{-(\alpha+1)} \quad \text{on} \quad [L_{min}, L_{max}]$$

α = 1.7 for consumer internet. Refitted quantiles matched to 0.00% at p50, p90 and p99.

Joint uplink / downlink rate
A 16-point lattice from a Gaussian copula, so the two directions are correlated but not proportional.

$$(r^{\mathrm{UL}}, r^{\mathrm{DL}}) \sim \mathrm{Lattice}_{16}(\rho) \quad , \quad \rho = 0.58$$

Measured Spearman rank correlation 0.546. The finite lattice keeps the number of distinct cohorts bounded.

Population conservation
Mobility moves people between areas without creating or destroying them.

$$\Sigma_k P_k(t) = 16\,000\,000 \quad \text{for all } t$$

Verified exactly — integer transitions, no rounding drift over 2 880 steps.

WHY BOTH MODELS EXIST

The 16-week training corpus was generated before the realism layer was built, so every throughput number in this deck comes from the simpler generator. The realism layer is validated and ready, but regenerating 112 days of history with it is a fresh multi-hour cluster run — one of the first items in the simulator backlog.

STRESS

Daily cycles, surges and failures

We model three kinds of stress and keep them independent. If surges and failures always happened together, a controller could score well just by spotting one and assuming the other.

DAILY CYCLE

Traffic rises and falls over the day and the week. Residential and business zones peak 7 to 11 hours apart, so the network never sees one single national peak. Weekends change both the height and the shape of the curve.

EVENT SURGE — A STADIUM MATCH

Six phases, each multiplying the normal arrival rate: crowd arrives ×1.5, kick-off ×2.1, match ×1.7, half-time uploads ×3.2, final whistle ×2.6, then a long exit tail. Upload-heavy services react hardest. Some events are announced two hours ahead; others are deliberately kept a surprise.

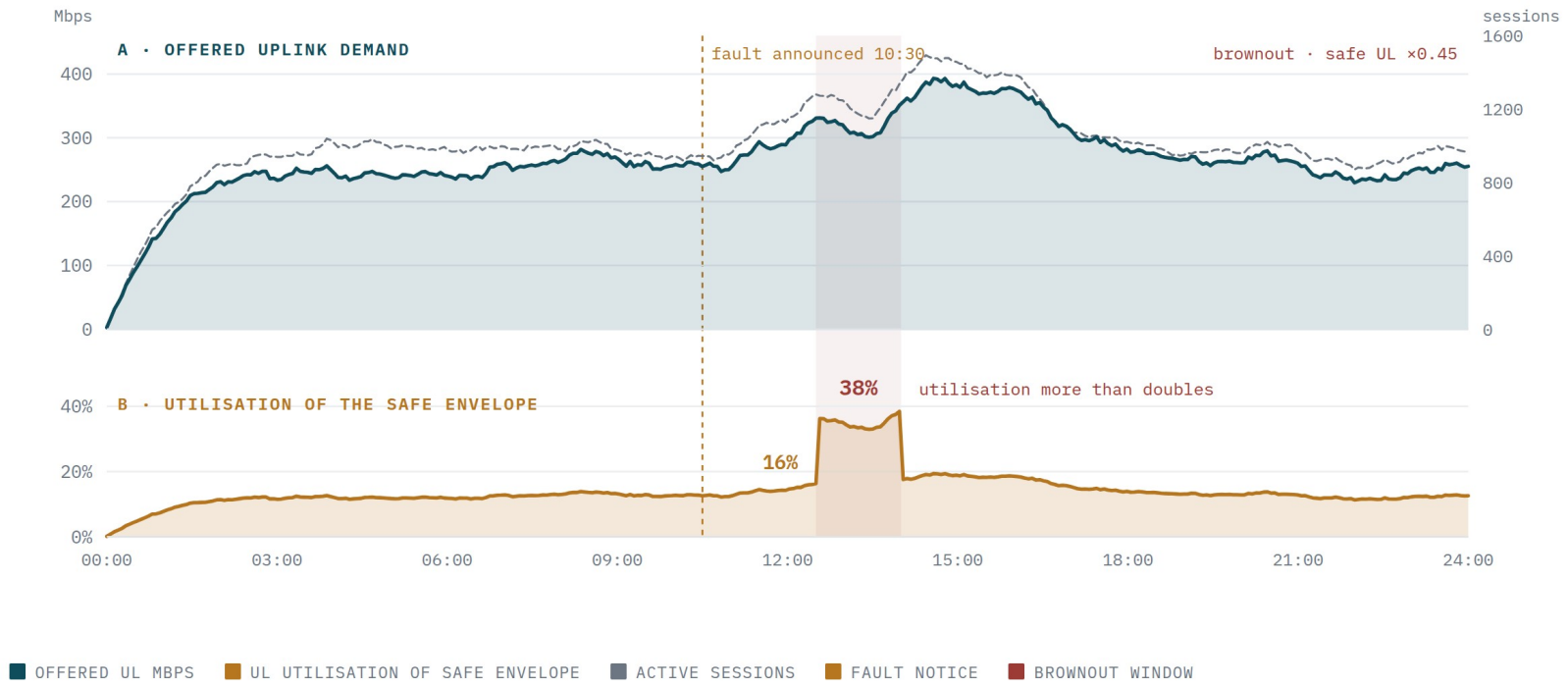
EQUIPMENT FAILURES

A box can lose part of its capacity — down to 45% — go almost fully offline (we leave 1% so the maths stays finite), suffer extra network delay, or restart. Recovery windows are modelled too.

×3.2	×0.45	1%	2 h	0
HALF-TIME UPLOAD SURGE	BROWNOUT CAPACITY LEFT	NEAR-TOTAL OUTAGE CAPACITY LEFT	NOTICE FOR A PLANNED FAILURE	NOTICE FOR A SURPRISE SURGE

ONE SIMULATED DAY

Demand, capacity, and a planned brownout



- **What the top panel shows**

  Uplink demand on a Delhi-NCR edge box, rising through the evening as the stadium fills.

- **What the bottom panel shows**

  How close that box is to its safe limit — the number the controller actually reacts to.

- **The brownout at 12:30**

  Safe uplink capacity drops from 2 028 to 912.6 Mbps for 90 minutes.

- **Why it matters**

  Demand barely changes. But headroom halves, so utilisation jumps from 16% to 38%.

- **The controller knew at 10:30**

  Exactly two hours of warning — enough time to stop sending new sessions there.

VERIFICATION

We test the simulator against itself

Claiming a simulator is realistic is easy. We check it instead: take its own output, re-fit every distribution from that output, and compare against what we asked for.

0.0 bytes	0.8198	0.00%	0.31%	EXACT	0
TRAFFIC THAT DOESN'T ADD UP	MEASURED DRIFT WE ASKED FOR 0.8200	SESSION-LENGTH ERROR	BANDWIDTH ERROR	PEOPLE COUNTED IN AND OUT	SESSIONS SENT TO A BANNED OR DEAD BOX

ONE SIMULATED DAY, IN BYTES

A single day carries 377.2 TB down and 113.8 TB up.

Of that, 239.6 GB down and 81.2 GB up are dropped. Nothing is rejected, and nothing is left stuck in a queue at the end of the day.

Every byte is accounted for. The maximum discrepancy across the whole run is exactly zero.

WHAT WE ARE NOT CLAIMING

The shapes come from published 3GPP standards and public measurement studies — how many service classes exist, how a stadium crowd behaves, how session lengths are distributed.

The absolute numbers — Mbps per session, box capacity, queue size — are our own assumptions.

Honest claim: standards-based, statistically verified synthetic traffic at national scale, not yet matched to real C-DOT measurements.

DATA QUALITY

We deliberately break the telemetry

A model trained on perfect counters will fail the moment it meets a real network. So the measurement channel is generated separately from the truth, and we inject faults on purpose.

0.8%	0.4%	0.08%	0.04%
READINGS THAT GO MISSING	READINGS THAT ARE STALE	COUNTER RESETS	PROCESS RESTARTS

The clean-up layer then has to earn its numbers back. It never bridges across a counter reset, and it discards impossible values rather than smoothing them into something that merely looks plausible.

OUR BIGGEST PROCESS MISTAKE — CAUGHT BY THE CHECKS, NOT BY LUCK

Two complete rounds of forecast training data were thrown away because telemetry quality was averaged over a whole time bucket instead of using the latest reading from the relevant boxes — which is what the live system actually does. Eleven cluster jobs were cancelled. No protected test data was touched. We restarted from version 3.

SCALE

The numbers at full size

16 weeks of history at 30-second resolution: 112 days, 322 560 steps, 16 128 decision rounds.

THE CONFIGURATION

ITEM	VALUE
Time simulated	16 weeks / 112 days
Step size	30 s
Total steps	322 560
Decision round	10 min / 20 steps
Decision rounds	16 128
Training rows	1 548 288
Per-box records	7 741 440
Arrival counts stored	30 965 760
Box × group records	9 289 728
Scheduled events	258 656
Sessions routed	≈ 4.39 billion
Output data size	≈ 9.6 GiB
Estimated run time	9 h 31 m

ONE MEASURED DAY

MEASUREMENT	VALUE
New sessions started	39 156 949 / day
Average arrival rate	453 sessions/s
Sessions active at day end	12 758 353
Steady-state active sessions	13 422 319
Average session length	8.23 h
Uplink, average / peak	3.22 / 4.09 Tbps
Downlink, average / peak	8.15 / 15.34 Tbps
Actually carried, up / down	3.18 / 8.03 Tbps
Total data offered	122.84 PB / day
Total data carried	121.08 PB / day
Largest queue, up / down	8.94 / 17.88 GiB
Run time for one day	5 m 06 s
Peak memory used	602 MiB

KEY NUMBERS · THROUGHPUT

How much traffic actually moves

All figures are measured from the completed seed-20260805 calibration day at the 16-million-user scale. “Offered” is what the users asked for; “carried” is what the network actually delivered.

11.21 Tbps	3.18 Tbps	8.03 Tbps	19.43 Tbps	121.08 PB	1.43%
CARRIED, INSTANTANEOUS AVERAGE, BOTH DIRECTIONS	CARRIED UPLINK	CARRIED DOWNLINK	PEAK OFFERED BOTH DIRECTIONS	CARRIED PER DAY	OFFERED TRAFFIC NOT DELIVERED

MEASURE	UPLINK	DOWNLINK	BOTH DIRECTIONS	NOTE
Offered / demand, average	3.22 Tbps	8.15 Tbps	11.37 Tbps	what users asked for
Carried, instantaneous average	3.18 Tbps	8.03 Tbps	11.21 Tbps	what the network delivered
Offered / demand, peak	4.09 Tbps	15.34 Tbps	19.43 Tbps	highest 30-second step of the day
Peak-to-average ratio	1.27×	1.88×	1.71×	downlink is far burstier
Delivered fraction	98.76%	98.53%	98.57%	the rest is dropped at an overloaded box
Aggregate daily volume, offered	—	—	122.84 PB / day	
Aggregate daily volume, carried	—	—	121.08 PB / day	1.76 PB / day lost

Consistency check: 121.08 PB per day × 8 bits ÷ 86 400 s = 11.21 Tbps. The daily aggregate and the instantaneous average agree exactly, which is what you want before trusting either of them.

What that means per session, per box, per step

Everything below is derived from the measured figures on the previous slide. The arithmetic is shown so it can be checked.

PER SESSION	VALUE	HOW IT IS DERIVED
Sessions started per day	39 156 949	measured
Session arrival rate	453.21 / s	39 156 949 ÷ 86 400
New sessions per 30-second step	13 596	453.21 × 30
Average session length	8.23 h	class-weighted mean
Sessions active at any moment	13.42 M	Little's Law: 453.21 × 29 628 s
Data carried per session	3.09 GB	121.08 PB ÷ 39.16 M
Throughput per active session	0.835 Mbps	11.21 Tbps ÷ 13.42 M
— uplink share	0.237 Mbps	3.18 Tbps ÷ 13.42 M
— downlink share	0.598 Mbps	8.03 Tbps ÷ 13.42 M

PER BOX AND PER STEP	VALUE	HOW IT IS DERIVED
Data carried per 30-second step	42.04 TB	121.08 PB ÷ 2 880 steps
Carried per UPF, average	467 Gbps	11.21 Tbps ÷ 24 boxes
— uplink per UPF	132.5 Gbps	3.18 Tbps ÷ 24
— downlink per UPF	334.6 Gbps	8.03 Tbps ÷ 24
Session setups per UPF	18.9 / s	453.21 ÷ 24
Active sessions per UPF	559 000	13.42 M ÷ 24
Fleet rated capacity	20.48 / 46.72	UL / DL across 24 boxes
Fleet safe capacity	15.83 / 36.17	after 75–80% safe
Utilisation of safe capacity	20.1% / 22.2%	uplink / downlink

THE NUMBER THAT EXPLAINS THE WHOLE OPTIMIZER RESULT

Across the fleet, the network runs at about 20% of its safe capacity. There is no aggregate shortage of capacity anywhere in this system. Every overload event is local and temporary — one box, during one surge or one failure. That is why a controller that redistributes load has so little to win, and why the failure-knowledge result in section 03 matters more than any demand forecast.

PERSPECTIVE

Is this really C-DOT scale? Partly.

COMPARISON	OUR SIMULATION	PUBLIC C-DOT FIGURES	WHAT IT MEANS
Users / subscribers	16 M nominal, ≈13.4 M active	≈12 million subscribers	Same order of magnitude — but sessions are not subscribers
Data per day	121.08 PB carried	more than 3 PB	We are about 40× higher
Network granularity	8 logical zones	75 000+ radio sites	Ours is far coarser
UPF boxes	24 synthetic	not published	Cannot compare
Per-box capacity	up to 1.92 / 3.84 Tbps	not published	Our own stress assumption
Telemetry format	our own schema	not published	Must come from C-DOT

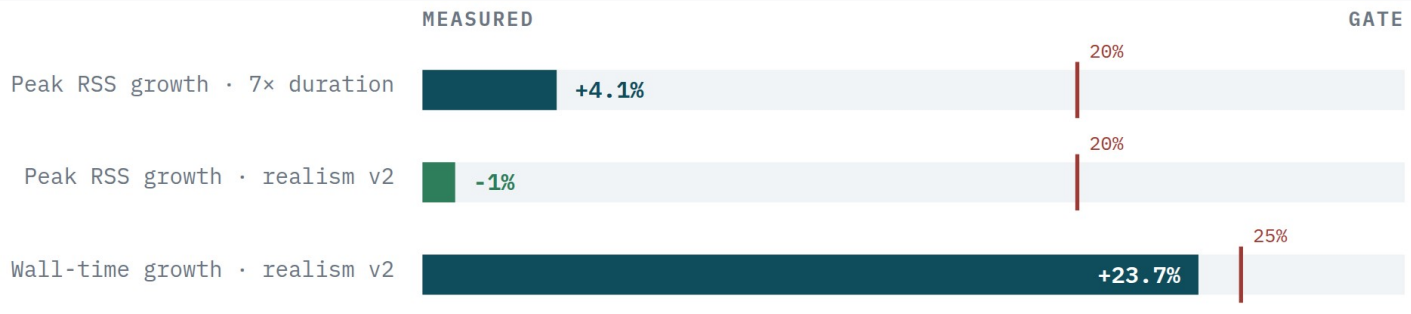
SO WHAT SHOULD WE SAY?

Our session count is in the same range as one public national deployment figure. Our traffic volume is much higher and our network detail is much coarser. The defensible phrase is “national scale”, not “C-DOT scale”.

SCALING UP · STAGE 1

Prove memory stays flat

A 112-day run produces far more data than fits in memory. If the simulator accumulated it, the run length would be capped by RAM. It does not — and this is the measurement that proves it.



- **Seven times the length**
  used only 4.1% more memory, against a 20% limit.
- **The richer traffic model**
  added 23.7% run time and slightly less memory. Both pass.

WHAT RSS MEANS

RSS — Resident Set Size — is how much physical RAM a process is occupying right now. It excludes anything swapped out to disk, so it is the honest measure of what a worker costs the node.

We read it from the Linux kernel directly: the VmRSS field of /proc/<pid>/status, sampled by the parent for every worker. Swap comes from VmSwap alongside it — which is how we can state that swap stayed at exactly zero. “Peak RSS” is the highest value seen across the run.

WHERE THE DATA ACTUALLY GOES

The simulator keeps at most one buffer of step rows in memory — 4 096 rows at the largest setting. When it fills, the buffer is written out as a compressed Parquet segment and cleared. Memory therefore tracks the buffer size, not the length of the run.

Those segments are written to scratch disk on the node the worker is already running on — not streamed over the network. Only when the shard finishes is the result copied once to the shared cluster filesystem, with a metadata file written last as the commit marker.

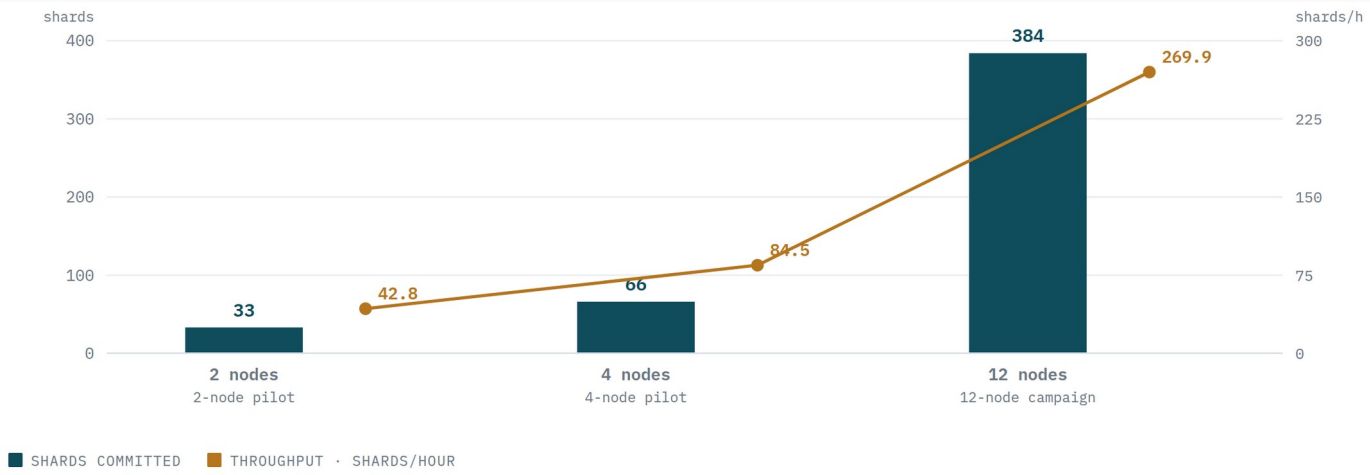
How many workers fit on one node?

Each node has 125 CPUs and 125 GB of RAM. We tried four settings, three times each, and required at least 70% CPU efficiency.

- **8 workers passed**
  at 71.2% CPU efficiency — the only setting that fully cleared the bar.
- **16 and 32 failed**
  at 67.0% and 68.6%, both below 70%.
- **64 never finished**
  all three required repetitions.
- **Memory was never the problem**
  even 32 workers used under 16% of the node's RAM. Something else is serialising.

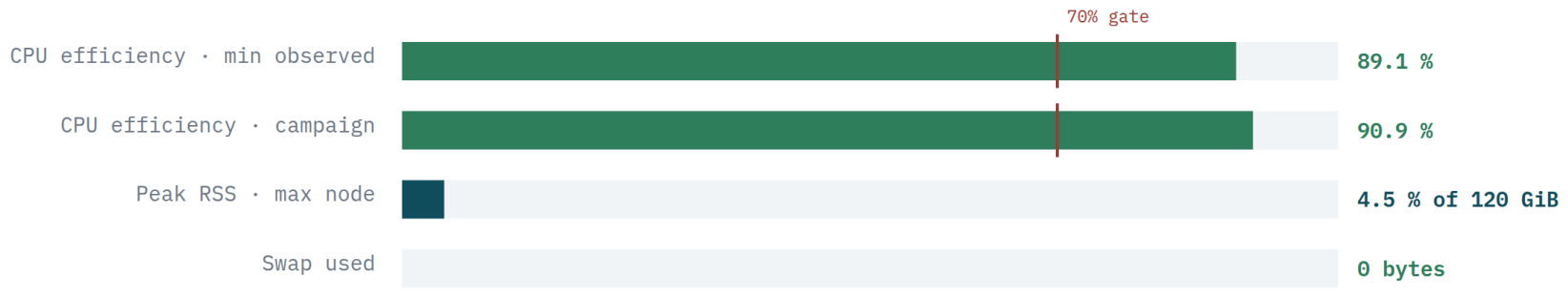
From 2 nodes to 4 to 12

Each node works on its own slice of the job list. Moving to 4 nodes requires a clean 2-node report; going beyond 4 requires a clean 4-node report on the exact same inputs — so a small unrelated test can never unlock the big expensive run.



12 NODES USED	384 JOBS COMPLETED	85.4 min TOTAL TIME ON 12 NODES
90.9% CPU EFFICIENCY WORST NODE 89.1%	5.4 GiB PEAK MEMORY OUT OF 120 GIB	0 CRASHES, SWAPS OR LOST JOBS

How much headroom was left



Every node passed every check. Peak memory never went above 4.5% of what was allocated, and the machine never had to swap to disk. That is the direct payoff of the memory work in stage 1.

- **What this does not measure**

  This is campaign throughput — how fast we can run experiments. It is not a measurement of how fast a live control loop would respond.

- **Where the ceiling is**

  Nothing in the design caps it at 12 nodes. We just need a clean 12-node report to unlock 24 or 48.

NEGATIVE RESULTS

What did not work on the cluster

About half the scale-up effort went into things that failed. Each one changed the design.

WHAT WE TRIED	WHAT HAPPENED	WHAT WE DID INSTEAD
Asking for local scratch disk	REJECTED — C-DOT's scheduler has no such resource	Fall back to the node's own /tmp with an operator-supplied disk budget
Requesting memory separately	REJECTED — refused when node placement is also specified	Put memory inside the node request instead
16, 32 and 64 workers per node	FAILED — 67.0% and 68.6% CPU; 64 never completed	Locked in 8 workers per node as the only setting that fully passed
A quick 30-minute test on 2 nodes	NOT USABLE — 6.8% CPU efficiency, far too short	Kept only for checking scheduling and file writing; never for choosing settings
Development controllers at full scale	BLOCKED — those settings are development-only	A controller must pass a fresh held-out evaluation before running on more than 4 nodes
Forecast training data v1 and v2	DISCARDED — telemetry quality averaged wrongly	Cancelled 11 jobs, kept the data for audit only, regenerated as version 3
Re-running the baseline every time	WASTEFUL — every candidate re-simulated the baseline	Fix planned: compute each baseline once. Saves about half the compute

NEXT STEPS

Bigger, and closer to a real operator

MAKING IT BIGGER

- **Stop re-running the baseline**
  Roughly half of every campaign is duplicated work. Computing each baseline once frees about 50% of the compute immediately.
- **Re-open the worker-count question**
  16 and 32 workers failed on CPU, not memory. Profiling the file-writing path and thread settings is the highest-value next step.
- **Beyond 12 nodes**
  Nothing in the design caps it. We need a clean 12-node report to unlock 24 or 48.
- **A separate analysis layer**
  Output files are immutable and hashed, so another system can read them without becoming a dependency.

MAKING IT MORE REALISTIC

- **Finer geography**
  8 zones cannot represent 75 000 radio sites. A cell-cluster layer with a realistic map of which box serves which area is the biggest gap.
- **Packet and radio behaviour**
  We model sessions, not packets, and ignore radio scheduling. Specified but not yet built.
- **Independent uplink and downlink**
  In the older data both were derived from session count, which is why all three forecast targets score identically.
- **Real telemetry from C-DOT**
  Metric names, scrape interval, counter behaviour, box capacities. All calibration is blocked on this.
- **Can sessions be moved?**
  The single most important open question — see section 03.

02

THE FORECASTER

Predicting how much traffic is coming

Not one prediction — 96 traffic groups, three quantities each, eight time horizons each. And the optimizer does not just want a best guess. It wants a safe upper bound it can plan against. So we measure accuracy and reliability separately.

METHOD

How the forecaster works

01 **What we predict.** For each traffic group, in each 10-minute window: how many new sessions start, and how much new uplink and downlink bandwidth they bring. Only new demand — because only new sessions can be steered.

02 **One model per horizon.** Separate models for 10, 20, 30, 40, 50, 60, 70 and 80 minutes ahead — 2 304 small models in total. We predict each horizon directly instead of stepping forward repeatedly, so errors do not pile up.

03 **Only past information.** Nine inputs: last value, a 6-window average, recent trend, yesterday's value at the same time, and sine/cosine terms for time of day and day of week. Anything not yet available at forecast time is rejected.

04 **Bias correction.** We measure the model's typical over- or under-shoot on separate data and subtract it.

05 **The safe upper bound.** Conformal prediction: look at how wrong the model was on held-out data, then add enough margin that ~90% (or 95%) of real values fall below the bound. The optimizer plans against the 95% bound.

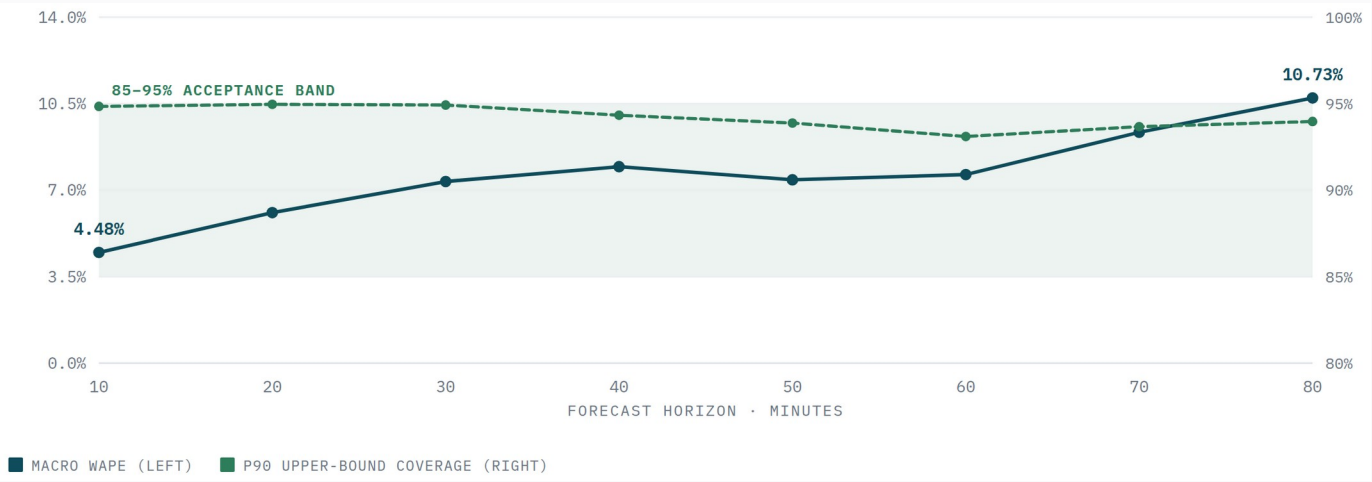
06 **Splitting data by time.** 70% train, 15% calibration, 15% test — always chronological, never shuffled. Each event is kept whole so no part of it leaks across the split.

RESULTS

Accuracy on 16 weeks of history

The main measure is WAPE — weighted absolute percentage error. Lower is better. 7.63% means the average prediction is off by about 7.6% of the true traffic volume.

7.63%	94.21%	96.69%	2 304	1.55 M	44.4%
AVERAGE ERROR ON UNSEEN DATA	REALITY STAYED UNDER THE 90% BOUND	SAME, FOR THE 95% BOUND	MODELS TRAINED	TRAINING ROWS	BETTER THAN "SAME AS YESTERDAY"



- **Further ahead is harder**
  Error rises from 4.48% at 10 minutes to 10.73% at 80 minutes.
- **But the safety bound holds**
  It stays inside its 85–95% target range at every horizon.
- **That second property is the one that matters**
  The optimizer needs a bound it can still trust even where the point estimate gets shaky.

COMPARISON

Against two simple baselines

Same test rows, and each baseline is also restricted to information available at its own forecast time — so this is a fair fight.

METHOD	AVERAGE ERROR	RIDGE IS BETTER BY	@10 MIN	@20 MIN	@80 MIN
Our model (calendar ridge)	7.63%	—	4.48%	6.09%	10.73%
Average of last 6 windows	14.30%	46.68%	7.58%	9.46%	20.84%
Same time yesterday	13.71%	44.36%	21.82%	21.82%	21.88%

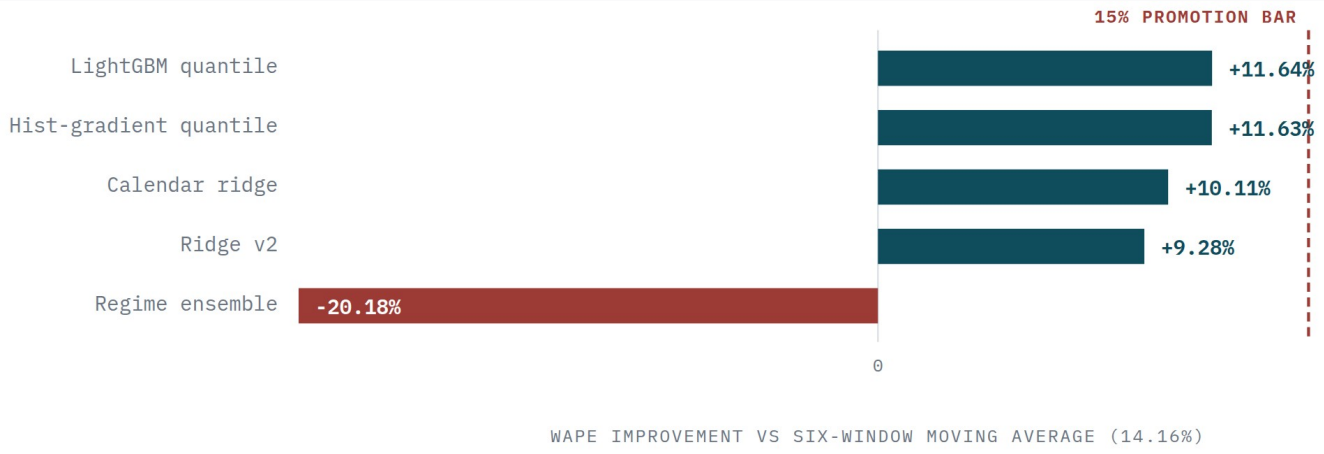
ONE CAVEAT WE SHOULD BE UPFRONT ABOUT

Sessions, uplink and downlink all report the same 7.63% error. Not because the model learned three things well — because the older simulator calculated uplink and downlink by multiplying session count by a fixed number. They are the same series in different units. The newer traffic model gives every session its own bandwidth draw, and it needs to become the default before we claim anything about predicting throughput.

MODEL COMPARISON

Five model families, head to head

We set the bar before running anything: beat the best simple baseline by 15%, keep the safety bound inside 88–95%, improve peak underprediction by 20%, and let no slice of the data get more than 5% worse.



- **The two boosting models won**
  and still missed the 15% bar by about 3.4 percentage points.

- **The regime-switching model is the interesting failure**
  It detects a surge and switches models. Best surge performance by far — 64% better at catching peaks — but 601% worse on its worst data slice and 20% worse overall.

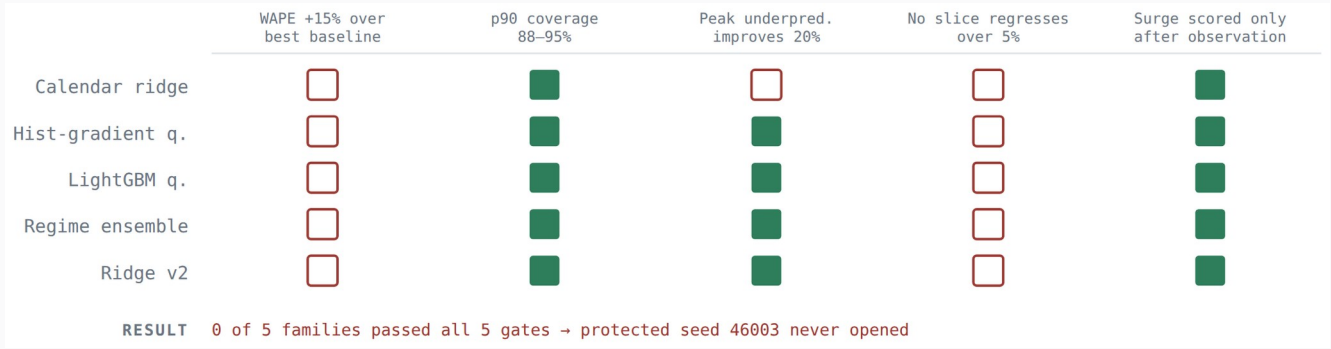
- **Tuning hard for the rare event**
  wrecked the ordinary case.

MODEL COMPARISON

Every family, every check

MODEL FAMILY	ERROR	VS BASELINE	BOUND HOLDS	PEAK UNDERPRED.	WORST SLICE	RELEASED?
LightGBM	12.512%	+11.64%	93.53%	+27.5%	+14.70%	No
Histogram gradient boosting	12.513%	+11.63%	93.53%	+27.4%	+14.34%	No
Calendar ridge (current)	12.729%	+10.11%	93.60%	+11.9%	+11.39%	No
Ridge v2	12.846%	+9.28%	94.39%	+25.8%	+12.42%	No
Regime-switching ensemble	17.017%	−20.18%	94.81%	+64.2%	+601.7%	No

384 models trained, 384 calibrated, 480 result files, plus 96 independent audit jobs that reloaded every model through the real production loading code.



NOBODY PASSED

No model has a full column, so none was released and the locked test data was never opened. That is the rule working as intended — not an experiment left unfinished.

FAILURE MODE

Where the forecaster actually falls over

The overall average hides the failure that matters. Same model, same day, split by what was happening at the time.

SITUATION	ERROR	PER-GROUP ERROR	90% BOUND HOLDS	95% BOUND HOLDS
Normal	6.51%	6.26%	94.19%	96.71%
Traffic surge	11.08%	11.02%	30.73%	37.85%
Box at reduced capacity	5.09%	5.73%	87.88%	91.96%
Box nearly offline	5.29%	6.57%	73.96%	78.13%
Network delay spike	5.04%	5.06%	92.36%	96.18%

DURING A SURPRISE SURGE, THE SAFETY BOUND IS RIGHT ONLY 30% OF THE TIME

Equipment failures change the network, not how much traffic people generate — so prediction accuracy barely moves. Surges are different. When demand suddenly jumps and nobody warned us, there is nothing in the past data that predicts it. The safety bound — the exact thing the optimizer plans against — is wrong precisely when the optimizer needs it most. This one row explains most of section 03.

COST

How long it takes to train and to run

TRAINING

3 minutes 3 seconds for the whole 16-week model set. Of that, only 13 seconds is actual maths — fitting 2 304 tiny nine-input models. Everything else is reading data off disk. Peak memory 1.8 GB, one node, no GPU.

A GPU would be useless here. The bottleneck is disk, not computation. That only changes if we move to large boosted-tree or neural models later.

The five-family comparison was a real cluster job: 384 trained models, 384 calibrated models, 480 result files and 96 audit jobs.

RUNNING IT LIVE

96 forecasts per decision round — one per traffic group — handed straight to the optimizer. In the one-day test the controller used a safe fallback for the very first round, then produced forecasts normally for the remaining 143.

Forecasting is never the bottleneck at any scale we tested. Every speed problem in this project is in the optimizer, not the model.

Model files are checksum-verified on load. If a model was trained on different traffic groups than the scenario expects, the system refuses to start rather than quietly using the wrong thing.

DECISION

Which model we use, and why

We run the calendar ridge model with conformal bounds. Not because it is the most accurate — LightGBM beats it by about 1.5 percentage points.

- **Nothing beat the release bar**
  The rule says the current model stays unless a challenger passes all five checks. None did. Keeping ridge is the required answer, not a preference.

- **Nine inputs, closed form, 13 seconds**
  Retrainable anywhere, easy to audit, and you can look at each coefficient and understand it.

- **Reliable where it counts**
  94.2% and 96.7% coverage on unseen data — the property the optimizer actually consumes.

- **The real gains did not carry through**
  LightGBM's 27.5% better peak-catching is genuinely useful, but it came with a 14.7% worse slice — and a better forecast did not produce better control.

Doubling the training history from 7 to 14 days almost halved the error, 16.02% → 8.70%. We then ran a full controller campaign with that better model: +6.72% average, but a confidence range from −3.32% to +18.21% that includes zero, −3.60% weighted by severity, and −22.98% on the worst day. Decision: stop before spending the compute.

NEXT STEPS

Forecasting

FIX THE EVALUATION FIRST

- **Use the intended data split**

  We used 70/15/15 instead of the specified weeks 1–11 / 12–13 / 14–16. A correctness fix, not a modelling change.

- **Report by situation as standard**

  Normal, surge, reduced capacity, offline and delay windows each reported separately before anything is released.

- **Break the uplink/downlink link**

  Give each session its own bandwidth so throughput prediction stops being a rescaled copy of session count.

THEN IMPROVE THE MODELS

- **Add weekly memory**

  A true one-week lag, multi-scale seasonal terms, rolling volatility, and features for events we already know are coming.

- **Pick the best model per group**

  Choose ridge, seasonal or LightGBM for each group and horizon — decided on validation data, never on test.

- **Share information across groups**

  Small groups have too little data alone. Pool them while keeping zone and service type distinguishable.

- **Go after the surge case directly**

  30% coverage during surges is the binding problem. Event calendars and a faster-adapting bound beat any base-model upgrade.

- **Neural models last**

  They add GPU cost and tuning effort, and will not fix a data-splitting or event-labelling problem.

03

THE OPTIMIZER

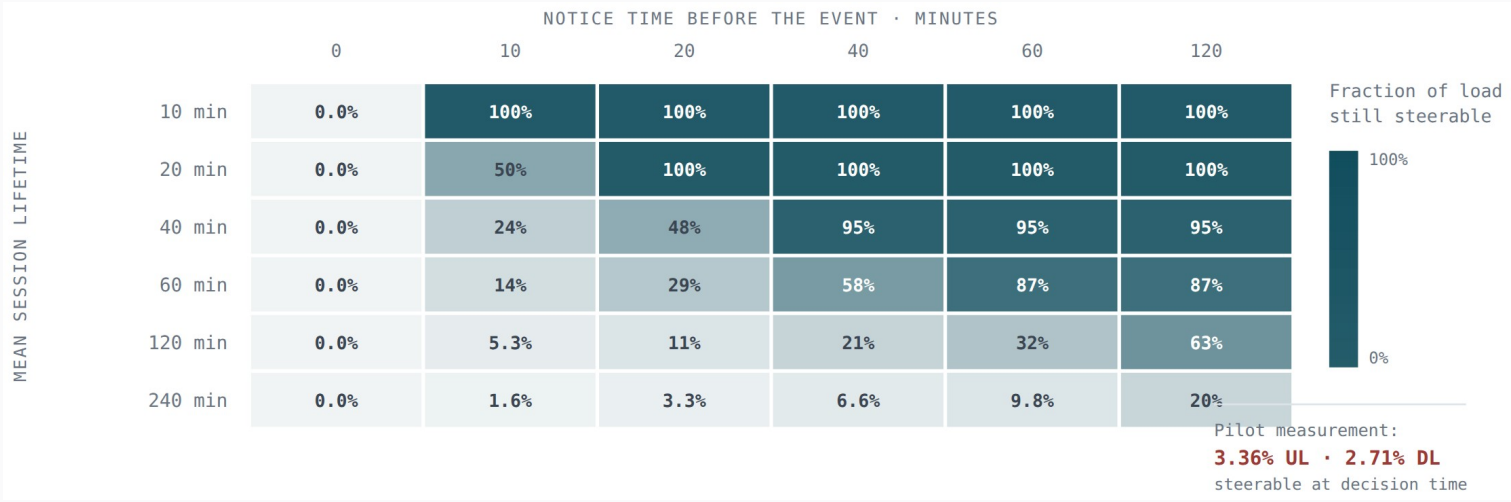
Deciding where traffic goes — and why it barely helps

Seven controller designs, 28 configurations, 588 head-to-head tests. The optimizer works: it solves, it validates, it acts. It just does not reliably beat a simple fixed rule. The reason turns out to be about the lever we have, not the algorithm.

THE CONSTRAINT

Start with the lever

The only thing a controller can change is the share of NEW sessions each box receives. Sessions already running stay where they are. That is not a simplification we chose — it reflects a genuinely open question about whether C-DOT's system can move a live session.



- **Read it like this**
  Columns are how much warning we get. Rows are how long an average session lasts.

- **The realistic corner**
  10 minutes of warning, 4-hour sessions → only 1.6% of load is still movable.

- **Measured on a real test day**
  3.4% of uplink and 2.7% of downlink traffic

THE AIRPORT OUTAGE, IN ONE NUMBER

One box went almost completely offline. That single event caused 103 965 of the controller's 105 181 total overload — 98.8% of it. All that traffic was already attached to the failed box, and a new-session-only lever cannot move it. Whatever the optimizer does with the remaining 1.2% simply cannot add up to a 20% improvement.

THE CANDIDATES

Seven controllers

CONTROLLER	HOW IT WORKS	STATUS
Simple (static)	Split traffic in proportion to each healthy box's capacity. No forecast, no memory.	IN PRODUCTION
Reactive	When a box gets busy, send less traffic to it.	Comparison
Forecast capacity	Simple rule, but adjust each box's headroom using the forecast.	Comparison
Linear program	Solve for the best split right now, given predicted demand and all capacity limits.	Tested
Cohort MPC	Plan two hours ahead. Track which sessions are running, when they end, what failures are coming.	Shadow only
Pre-drain	When we know a box is about to lose capacity, stop sending it new sessions in advance.	Shadow only
Oracle	Knows the whole day in advance. Solves the entire day at once.	NOT DEPLOYABLE

WHAT THE LINEAR PROGRAM ACTUALLY OPTIMISES

The unknowns are traffic shares per (group, box). The objective is dominated by a very large penalty on exceeding a box's safe limit, plus a smaller cost for general utilisation and small penalties for sending traffic out of zone or changing the plan too often. Constraints cover uplink and downlink capacity, session count, which boxes each group may use, box health, delay limits, and a cap on how much of one group can pile onto one box. If no valid answer exists, the solver says so rather than inventing one. Each solve is capped at one second.

EXPERIMENT 1

The linear program vs the simple rule

One busy simulated day: three four-hour local surges, a box at reduced capacity, a box nearly offline, a network delay incident, and recovery periods. All three controllers saw identical conditions.

MEASUREMENT	SIMPLE	REACTIVE	PREDICTIVE	PREDICTIVE VS SIMPLE
Uplink overload (main metric)	107 767 s	211 891 s	105 181 s	2.40% better
Downlink overload	59 906 s	118 167 s	56 376 s	5.89% better
Time spent overloaded, up	16 440 s	19 650 s	13 230 s	19.53% better
Time spent overloaded, down	30 480 s	7 200 s	7 200 s	76.38% better
Data dropped, up	49.27 TB	119.44 TB	49.16 TB	0.23% better
Data dropped, down	79.19 TB	163.05 TB	76.30 TB	3.66% better
Failed session setups	0	0	0	no change

- **Reactive is worse than doing nothing clever**
  50% more overload and 59% more dropped data than predictive. So the real competition is the simple rule.

- **2.40% against a 20% target**

  The predictive controller did beat the simple rule — by nowhere near enough.

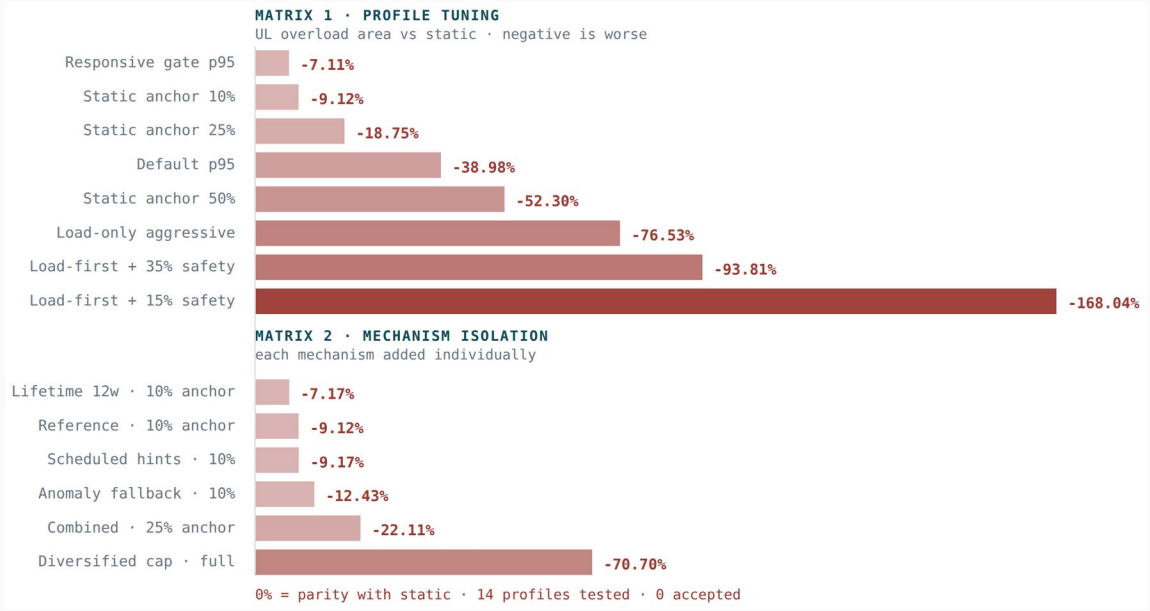
- **It mostly chose not to act**

  It left the plan unchanged in 109 of 143 rounds, acting only 34 times.

EXPERIMENT 2

Tuning made it worse, every time

Eight settings across two validation days, then six more that each added one specific capability. Negative means worse than the simple rule.



WHY THE OPTIMIZER ACTIVELY HURTS

It correctly reduces the peak inside its planning window by packing traffic onto whichever boxes look free right now. But sessions last hours. Those sessions become a permanent load that a new-session-only lever cannot undo later. The simple rule spreads every group across all its allowed boxes, which is far more robust. We confirmed it directly: several settings reduced immediate overload while making total overload and dropped data worse over the full day.

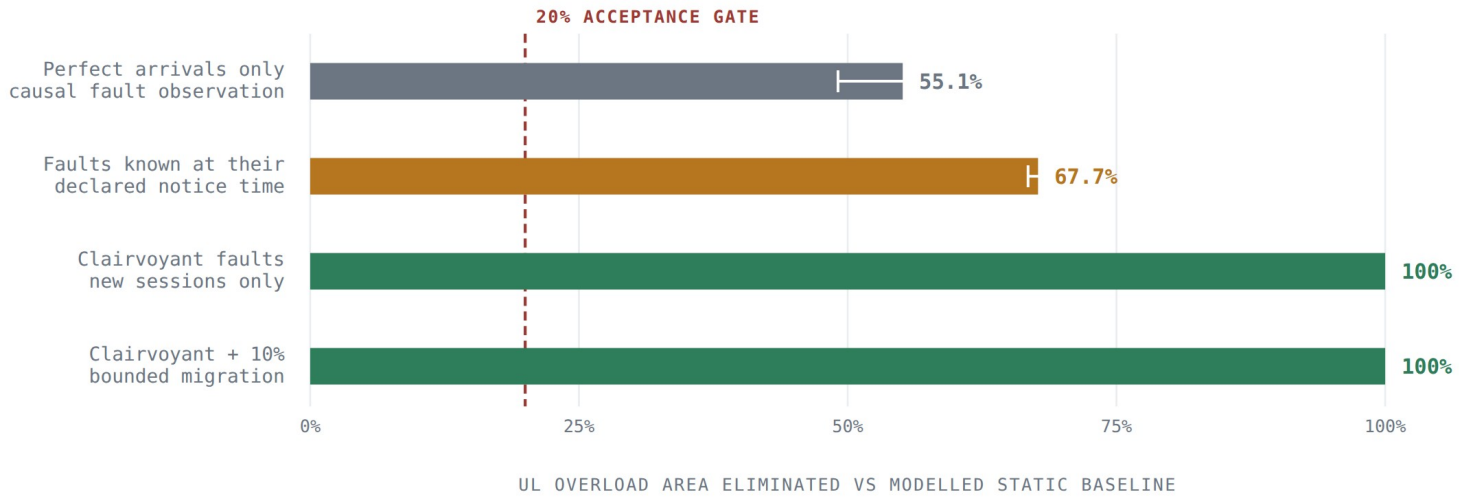
We then tested whether any single missing capability explained the gap: telling the optimizer about scheduled events, adding a surge detector, weighting by how long sessions last, and capping how much one box can take. All six variants still lost.

Session-length weighting helped a little — 1.95 percentage points against a 9-point deficit. The conclusion was explicit: the next change is not another parameter, it is a different kind of model.

EXPERIMENT 3

How much room for improvement is there?

Before building something harder, we asked whether the problem is solvable at all. An oracle reads the exact arrivals for the whole day, then solves the entire day at once. It cannot be deployed — it knows the future — but it tells us the ceiling.



- **Failures matter more than demand**

  Knowing all future arrivals removes ~55% of overload. Knowing about failures in advance removes 100%.

- **That reframes the whole problem**

  We had been building better demand forecasts when the leverage was in failure knowledge.

- **The middle rows are unstable**

  One day gave −91%, the other +97%. Before an unannounced failure many plans look equally good, and the solver picks between them almost at random.

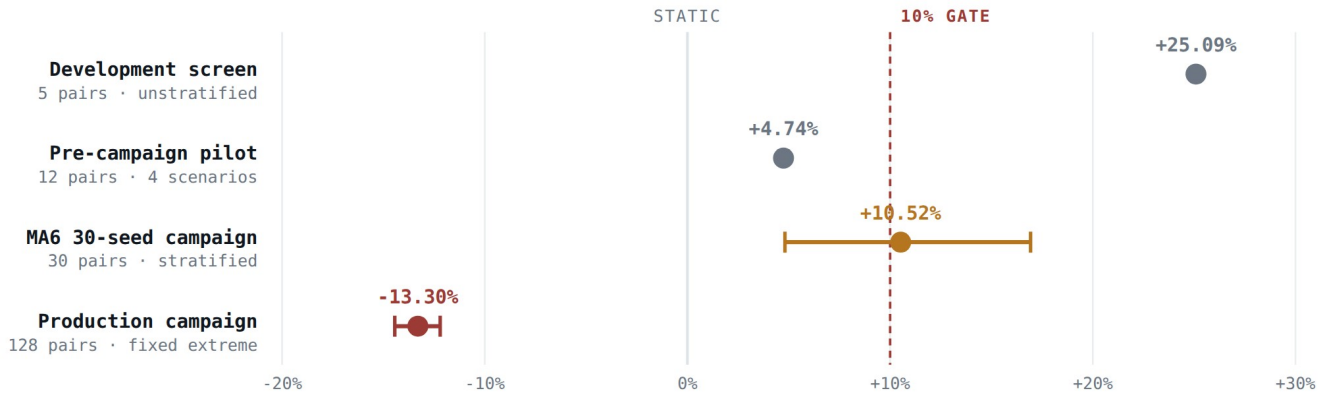
- **Which is itself evidence**

  The next controller needs a reason to prefer staying close to the simple rule, and it needs to care about where it leaves itself exposed later.

EXPERIMENT 4

MPC, in four rounds

MPC means Model Predictive Control: plan several steps ahead, act on the first step, re-plan with fresh information. Ours tracks every running session over a two-hour horizon, and only publishes a plan if it can prove that plan beats the simple rule from the same starting state.



Every time we made the test stricter, the result got smaller — and on the realistic production test set it flipped negative.

ROUND	DAYS	AVERAGE GAIN	WORST DAY	VERDICT
First look	5	+25.09%	+22.87%	Promising
Wider test	12	+4.74%	−13.91%	Blocked
Full campaign	30	+10.52%	−23.50%	Demo only
Production test	128	−13.30%	−36.22%	Rejected

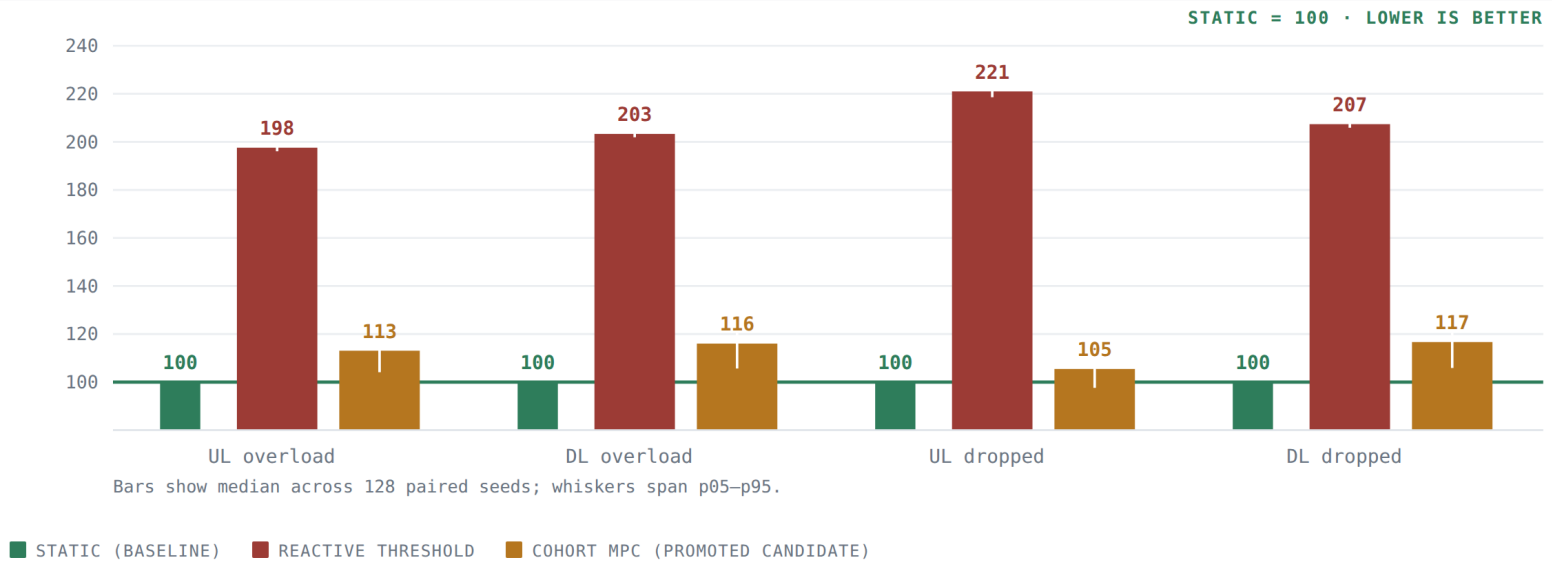
RECONCILING +10.52% WITH −13.30%

Both used the same controller, settings and metric. What differed was the mix of test days. The 30-day set was deliberately balanced across four stress types, and all the gain came from planned-failure days (+19% there) while surprise and mixed days were already slightly negative. The production set was a larger, different mix with no overlap. On it, only 2 of 128 days improved.

EXPERIMENT 4 · RESULT

128 production test days

The simple rule is set to 100 on every measure. Lower is better.

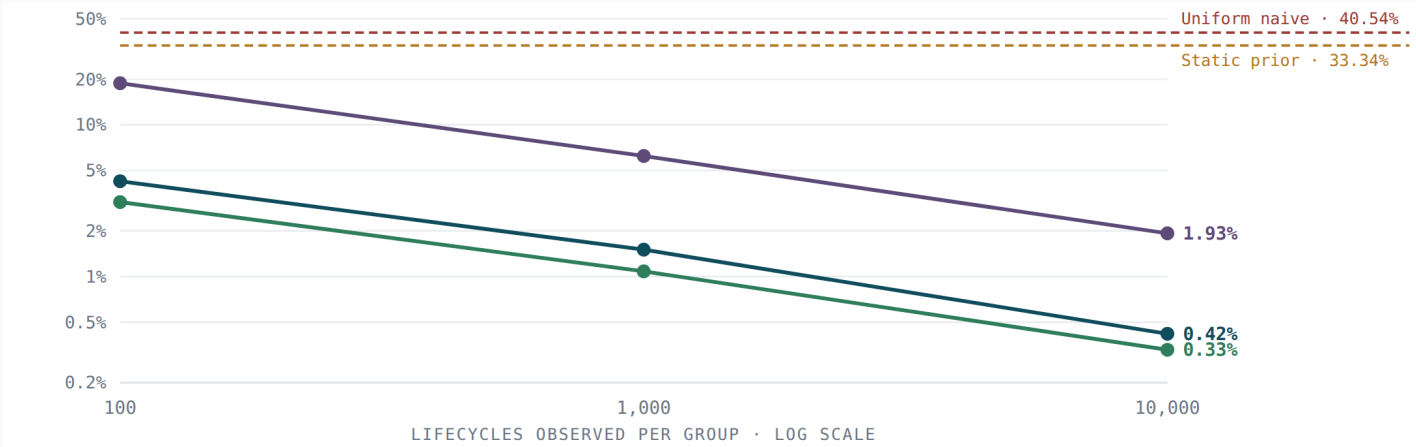


- **Reactive lands at 198–221%**
  on every single loss measure. It is the worst controller we tested.
- **MPC improves that to 105–117%**
  much better than reactive, and still consistently worse than doing nothing clever.
- **The simple rule wins on all four**
  uplink overload, downlink overload, uplink drops and downlink drops — simultaneously.

EXPERIMENT 5

Learning how long sessions last

MPC needs to know how long the sessions it places will occupy a box. Using the simulator's own setting would be cheating, so we estimate it from ordinary start/stop logs — using Kaplan–Meier, which handles the awkward part: many sessions are still running when we look.



- **More observations, better estimates**
  With 10 000 sessions per group the estimate is off by 0.42%. A fixed guess is off by 33.3%.
- **We tested it blind**
  25 trials against each of five distributions it was never told about. Average error 2.9–3.5%.
- **It handles drift**
  Drift mid-experiment raised error to 4.2%; re-estimating brought it back to 2.75%.

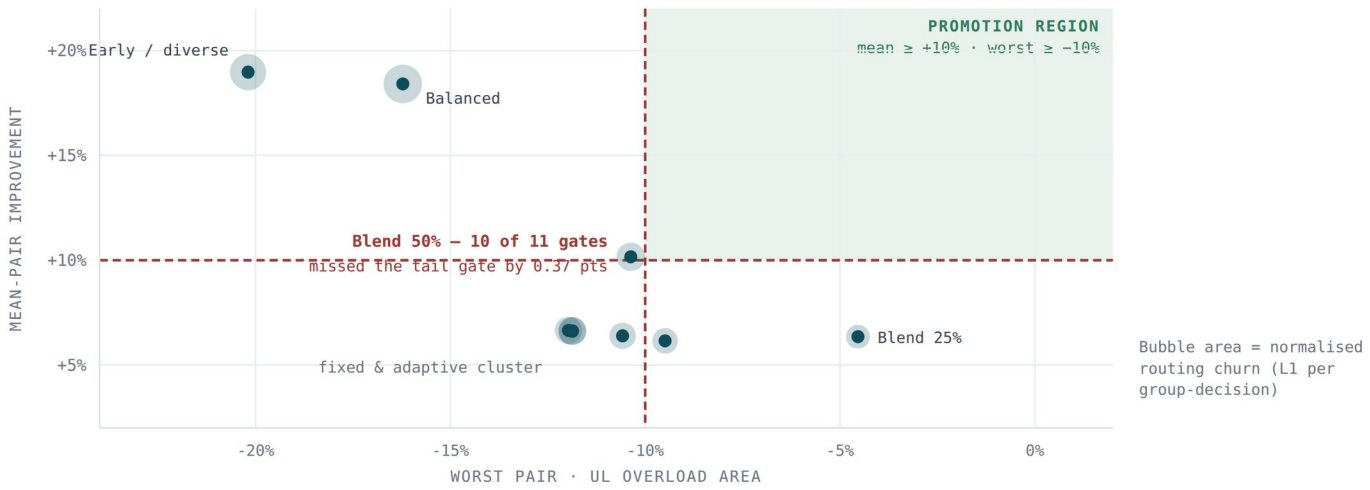
THE ESTIMATOR WORKS. IT JUST DOESN'T HELP.

Feeding these curves into MPC changed almost nothing. Across twelve test days: the perfect oracle curve gave +11.22%, the accurate estimated curve gave +9.91%, and a deliberately wrong uniform curve — off by 40.5% — gave the best result of all at +12.25%. In this setting, survival accuracy and control quality are essentially unrelated.

EXPERIMENT 6

Pre-drain, and an impossible trade-off

If failures matter more than demand, attack failures directly. Pre-drain does one thing: when we know a box will lose capacity soon, stop sending it new sessions ahead of time. Small problem — 648 unknowns — solves in about 30 milliseconds.



- **Each dot is one configuration**
  Average gain up the axis, worst-day loss across it. Bubble size is how much routing churn it causes.
- **The green box is the release zone**
  A configuration must land inside it. Nothing does.
- **Acting strongly**
  buys a good average and an unacceptable worst day.
- **Acting gently**
  is safe and gains too little.
- **The closest miss**
  50% strength passed ten of eleven checks and failed the worst-day limit by 0.37 percentage points. We did not relax the limit.

EXPERIMENT 6 · DETAIL

Every pre-drain configuration

240 test days on fresh data. A candidate must pass every check, not most of them.

CONFIGURATION	AVERAGE GAIN	95% RANGE	WORST DAY	PLANNED FAILURES	MIXED STRESS	SLOWEST DECISION	FAILED ON
Full strength	+18.41%	[+3.89, +35.43]	−16.22%	+79.20%	−5.58%	—	worst day, churn
Early & spread out	+18.97%	[+3.78, +36.63]	−20.19%	+83.05%	−7.18%	—	worst day, churn
50% strength	+10.16%	[+0.99, +21.92]	−10.37%	+43.89%	−3.27%	—	worst day only
25% strength	+6.35%	[+0.66, +14.19]	−4.54%	+26.76%	−1.37%	—	average only
Fixed 35%	+6.15%	[−0.84, +16.37]	−9.49%	+28.40%	−3.79%	764 ms	3 checks
Fixed 40%	+6.39%	[−0.99, +16.74]	−10.58%	+30.01%	−4.44%	225 ms	3 checks
Fixed 45%	+6.65%	[−1.12, +17.18]	−11.97%	+31.59%	−4.94%	970 ms	4 checks
Adaptive, early taper	+6.61%	[−1.15, +17.19]	−11.87%	+31.53%	−5.09%	739 ms	4 checks
Adaptive, late taper	+6.63%	[−1.15, +17.22]	−11.87%	+31.59%	−5.09%	628 ms	4 checks

WHY MAKING IT ADAPTIVE DIDN'T SAVE IT

We tried varying the strength based on how busy the box currently was. The mechanism worked — strength ranged from 25% to 50% as intended. It still failed on the same day. The known capacity loss entered the planning window first, and the surprise surge started well after that. At every pre-drain decision the box looked only 35% busy, so the controller applied full strength. By the time the surge arrived those sessions were already placed. Reacting to how busy things are right now cannot protect you from a surprise that arrives after you have already committed.

COST

How much compute a decision takes

We measured solver status, problem size, solve time and — importantly — the complete time to make a decision, not just the solver call. Earlier checks only looked at the solver, and that hid real outliers.

CONTROLLER	PROBLEM SIZE	SOLVE TIME	SLOWEST SOLVE	SLOWEST FULL DECISION	TIMEOUTS / ERRORS
Pre-drain	648 unknowns	27–28 ms typical	118–127 ms	225–970 ms	0 / 0
Cohort MPC	≈ 3 818 unknowns	4.1–4.2 s	6.18 s	5.4–6.2 s	0 / 0
Linear program	96 groups × 24 boxes	< 1 s (capped)	–	–	always solved

These worst-case times were measured while 120 simulations ran at once on a fully loaded 125-CPU machine. They are NOT a clean measurement of production speed and should not be quoted as

THE TIMEOUT DISCOVERY

Early MPC runs showed the solver timing out about 69% of the time. We removed the timeout to check whether the negative result was just an artefact of the optimizer never running. It ran fine — and the gap did not close. So the negative result is real.

But the 2-second solver limit was never a limit on the whole decision. One decision makes several solver and validation calls.

CAMPAIGN COMPUTE

Each 120-day campaign took about 1 hour 10 minutes on one exclusive 125-CPU node, running at roughly 12 000% CPU — about 120 cores busy at once.

Only the fixed 40% configuration met the new 500 ms decision budget, and it still failed on average gain, confidence and worst day. The speed check did its job: it exposed slow decisions that solver-only reporting had completely hidden.

THE FULL SCORECARD

Fifteen candidates, thirteen checks

A candidate must fill every square.

	CONJUNCTIVE GATES 01	02	03	04	05	06	07	08	09	10	11	12	13	GATES PASSED
MPC h3 configured-survival	☐	☐	☐	■	■	■	■	■	☐	■	☐	⬚	■	7 / 13
MPC h3 lifecycle lognormal	☐	☐	☐	■	■	■	■	■	☐	■	☐	⬚	■	7 / 13
MPC h6 lifecycle heavy-tail	☐	☐	☐	■	■	■	■	■	☐	■	☐	⬚	■	7 / 13
Pre-drain balanced	■	■	☐	■	☐	■	■	■	■	☐	■	⬚	■	9 / 13
Pre-drain early / diverse	■	■	☐	■	☐	■	■	■	■	☐	■	⬚	■	9 / 13
MPC h3 MA6 configured	☐	☐	■	■	■	☐	■	■	■	■	☐	⬚	■	8 / 13
MPC h3 MA6 heavy-tail	☐	☐	■	■	■	☐	■	■	■	■	☐	⬚	■	8 / 13
MPC h3 MA6 lognormal	☐	☐	■	■	■	☐	■	■	■	■	☐	⬚	■	8 / 13
Pre-drain blend 25%	☐	■	■	■	■	■	■	■	■	■	■	⬚	☐	10 / 13
Pre-drain blend 50%	■	■	■	■	☐	■	■	■	■	■	■	⬚	☐	10 / 13
Pre-drain adaptive 50–75	☐	☐	■	■	☐	■	■	■	■	■	■	☐	☐	8 / 13
Pre-drain adaptive 60–85	☐	☐	■	■	☐	■	■	■	■	■	■	☐	☐	8 / 13
Pre-drain fixed 35%	☐	☐	■	■	■	■	■	■	■	■	■	☐	☐	9 / 13
Pre-drain fixed 40%	☐	☐	■	■	☐	■	■	■	■	■	■	■	☐	9 / 13
Pre-drain fixed 45%	☐	☐	■	■	☐	■	■	■	■	■	■	☐	☐	8 / 13

01 Mean-pair UL improvement ≥ +10%
02 Bootstrap CI lower bound > 0
03 Severity-weighted improvement > 0
04 Unknown / mixed regression ≥ −2%
05 Worst pair ≥ −10%
06 No DL, drop or establishment regression
07 No solver timeout or error
08 Unexpected fallback ≤ 1%
09 Skipped decisions ≤ 95%
10 Normalised churn ≤ 0.05 L1
11 Measured empirical-survival robustness
12 End-to-end decision latency
13 Zero predicted overflow

■ GATE PASSED ☐ GATE FAILED — HOLLOW SQUARE, RED OUTLINE ⬚ NOT APPLICABLE — DASHED SQUARE

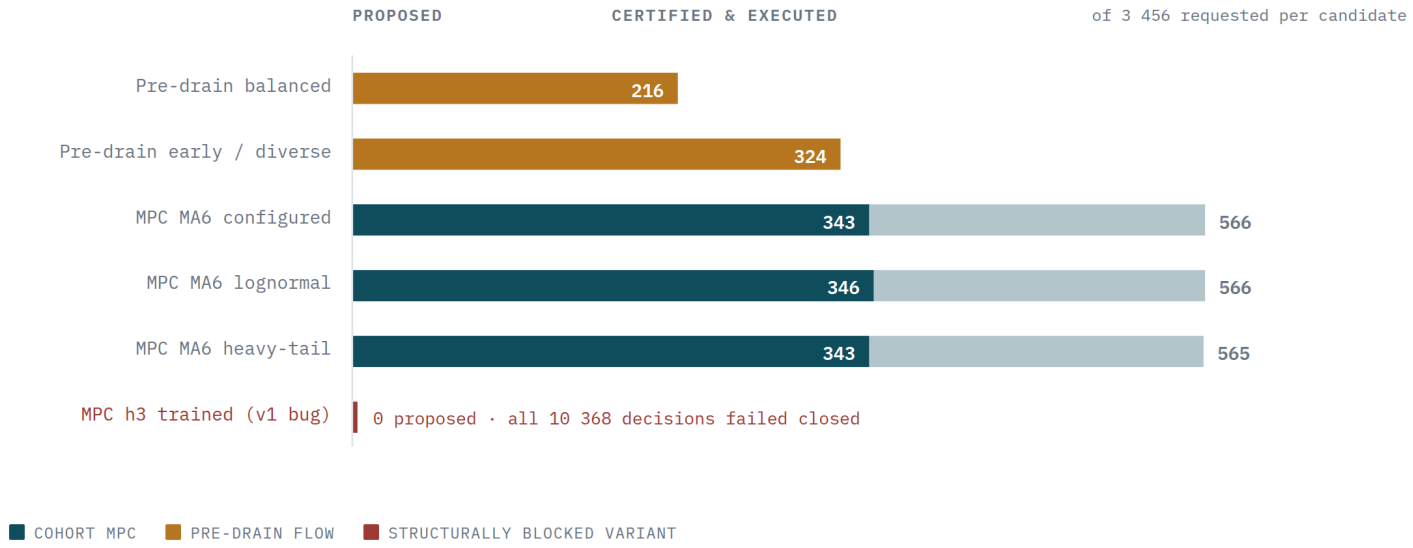
NOBODY FILLED A ROW

The first three MPC variants failed for a different reason entirely: the trained forecaster needed 144 completed history windows to warm up, and the experiment only had 143. So all 10 368 decisions safely refused to act — correct behaviour, and useless as evidence.

WHAT CHANGED

A history-compatibility preflight now rejects that interface before an experiment can consume a development seed pool.

What happened to every proposed action



- **MPC proposed 1 697 actions**
  across three variants, once the warm-up problem was fixed.
- **1 032 were executed**
  after passing the safety check.
- **665 were rejected**
  mostly because the improvement was not large enough to justify acting.
- **166 had no valid solution**
  and correctly fell back.
- **The machinery works end to end**
  The benefit is simply not there.

SELF-AUDIT

A safety problem we found in our own results

“SOLVED” DID NOT ALWAYS MEAN “VALID”

A later review found that seven pre-drain configurations produced answers the solver marked as solved while still slightly exceeding a capacity limit. Previously we would have called those plans feasible. They were not.

- **What changed**
  The system now reports any such excess explicitly.
- **A new mandatory check**
  “No capacity exceeded” is now a release requirement, not a nice-to-have.
- **Fail closed**
  Any violation hands control straight back to the simple rule.
- **What we did not do**
  We did not re-score the old experiments — their conclusions were negative anyway — but we no longer describe those actions as validated.

WHY REPORT THIS AT ALL?

Nobody outside the project would have found this. The affected experiments had already been rejected, so the headline conclusions do not change.

We report it because the value of this whole apparatus is that its claims can be trusted. An evidence system that quietly corrects its own record is worth more than one that never admits to a correction.

It also produced a real improvement: a check that did not exist before now runs on every candidate.

DECISION

What we run in production, and why

The simple rule. Split new sessions across each group's healthy boxes in proportion to their capacity. No forecast, no memory, no solver.

- **It wins on the metric we chose in advance**
  Across 128 production test days it is best on uplink overload, downlink overload, uplink drops and downlink drops — all four at once.
- **Spreading is structurally safer**
  When each group has three to six eligible boxes, sessions last hours, and you cannot move them afterwards, spreading beats concentrating.
- **It has no bad days**
  No worst-case loss, no validation to fail, no solver to time out, no data to go stale, no forecast to be wrong during a surge.
- **The clever controllers stay available, but supervised**
  MPC and pre-drain remain implemented and can run in shadow mode. They give up control the moment telemetry looks uncertain, their data is stale, or a capacity limit would be exceeded.

ONE SPECIFIC WARNING

The reactive controller is 98–121% worse than the simple rule on every production measure. “Move traffic away from the busy box” is the most intuitive policy and the worst one we tested. Whatever C-DOT deploys, it should not be that.

100	105–117	198–221
SIMPLE RULE BASELINE	COHORT MPC	REACTIVE CONTROLLER

NEXT STEPS

The optimizer

GET A BETTER LEVER

- **Settle the session-migration question with C-DOT**

  The oracle shows that moving just 10% of live sessions per 10 minutes removes all overload. If the system can do it, model its cost. If not, publish the ceiling honestly and stop optimising against it.

- **Add capacity as an action**

  Spinning up another box is a fundamentally different lever from redistributing existing ones. The simulator already supports it.

- **Reversible placements**

  Short-lived allocations that can be revised before they turn into hours of committed load.

CHANGE THE METHOD, NOT THE NUMBERS

- **Limit commitments under uncertainty**

  Instead of another blend percentage, give the controller an explicit uncertainty budget it may not exceed.

- **Validate against a full simulated day**

  Not just “is this step better” but “does this whole plan still beat the simple rule”, including forecast error.

- **Do nothing when nothing is wrong**

  The controller sometimes accepts a small uplink loss for a large downlink gain on days with no uplink problem. It should decline.

- **Design to the decision deadline**

  Treat 500 ms as a design constraint from the start, not something measured afterwards.

- **Reinforcement learning, later**

  Out of scope until the lever question is settled. An RL agent inherits exactly the same 1.6% of movable traffic.

04

CONCLUSION

The system knew when the algorithm wasn't ready

That is the real deliverable. A simulator, a forecaster and an optimizer are ordinary engineering. A test apparatus that can run 588 experiments, produce several genuinely exciting intermediate results, and still refuse to release any of them — that is the hard part.

EVERYTHING WE RAN

The complete inventory

588 CONTROLLER PAIRS · 28 DECLARED CANDIDATE CONFIGURATIONS

125 blind survival trials · 516 declared candidate pairs + 72 sensitivity comparisons · 0 protected seeds consumed

588	28	125	174	0	0
CONTROLLER COMPARISONS	CONFIGURATIONS TESTED	BLIND SURVIVAL TRIALS	CODE TESTS PASSING	PROTECTED TEST DATA USED	UNSAFE RELEASES

What is true, stated plainly

- **The simulator is solid engineering.**
  Traffic adds up to zero error, memory stays flat as runs get longer, every result is reproducible from a hash, and the final campaign ran 384 jobs on 12 nodes with no failures.

- **The forecaster and the survival estimator both work.**
  LightGBM improves every target and catches peaks 27.5% better. The survival estimator gets within 0.42% and survives five unknown distributions. Neither cleared its release bar.

- **The room to improve is real — but not where we were looking.**
  The oracle shows new-session steering can remove all overload, but only with advance knowledge of failures, not of demand.

- **No deployable controller captured it.**
  Under realistic mixed conditions, worst-day limits and speed limits, MPC was break-even to harmful and pre-drain traded average gain for unsafe worst days. The simple rule stays.

- **None of this is calibrated to real C-DOT traffic.**
  Every record is labelled synthetic. The next real milestone is a telemetry contract and capacity figures from the operator — not another model.

THE PATH FORWARD

Three questions that unblock everything

01

Can a running session be moved to another box?

Yes turns 1.6% movable traffic into a solvable problem, and makes every controller in this deck worth revisiting. No sets a hard, publishable ceiling on predictive steering — and tells us to spend the effort on capacity instead.

02

What does C-DOT's telemetry actually look like?

Metric names, scrape interval, counter behaviour, reset semantics, and real per-box capacity. Until we have these, “accuracy on C-DOT traffic” is not something we can measure.

03

Is uplink overload the right thing to optimise?

We chose it in advance and stuck to it, which is why these results can be trusted. But it is dominated by single catastrophic outages. Reporting an “avoidable overload” measure alongside it — never instead of it — is the natural next refinement.

The evidence system worked. That is what we are asking you to trust next.

Every figure in this deck comes from frozen result files in the project: the traffic realism evaluation, the forecaster freeze and baseline comparison, the five-family forecast selection, the three optimizer development campaigns, the survival campaign, the oracle bound evaluation, the MPC campaign results, and the final 12-node production metrics.

Nothing was re-scored to build this presentation, and no protected test data was used. All traffic data is synthetic.